

CS-412 Software Security: Fuzzing libpng

Louis Gogniat

Donia Gasmi

Srushti Singh

Khaled Kerouch

Abstract

Image parsing libraries have emerged as compelling targets for fuzzing due to their widespread deployment, continuous exposure to untrusted inputs, and long history of parser-driven memory-safety vulnerabilities. In this work, we present a coverage-guided fuzzing campaign against `libpng 1.2.52`, the reference C implementation of the PNG image format. Using `AFL++` with compile-time instrumentation and `AddressSanitizer`, we built a harness targeting the full PNG decoding pipeline, assembled a format-aware seed corpus, and ran both an instrumented and a QEMU-mode (binary-only) campaign for one hour each. Our standard campaign discovered 24 unique crashes, all caused by a `stack-use-after-scope` bug in `png_inflate()` (`pngutil.c`), reachable via both `iCCP` and `zTXt` chunk handlers. The bug was silently fixed in `libpng 1.2.53`.

1 Introduction

Fuzzing is one of the most effective techniques for finding memory-safety bugs in C libraries that parse untrusted binary input. We target `libpng` library which parses a richly structured binary format with CRC-protected chunks, zlib-compressed data streams, and a complex transformation pipeline, all of which constitute deep attack surface. Several historical CVEs in `libpng` were originally found by fuzzing, making it a well-motivated target. We intentionally chose version 1.2.52 over the latest release to increase the likelihood of finding real bugs while keeping the methodology sound. Finally, we examine the effect of persistent mode execution on fuzzing performance and discuss how instrumentation depth and sanitizer overhead influence overall campaign efficiency.

2 Q1 — Harness Design

2.1 Entry-Point Selection

We targeted the full PNG *decoding* pipeline via `png_read_info` followed by `png_read_image`. This call sequence exercises the complete parser: signature validation, chunk dispatch, IHDR field parsing, ancillary chunk handling (`iCCP`, `bKGD`, `tEXt`, `tRNS`), zlib decompression, and scanline defiltering. Several notable CVEs including CVE-2015-8126 and CVE-2016-10087 were triggered through this path, making it the most security-relevant entry point.

We considered two other alternatives. The *encoder* pipeline (`png_write_image`) would also have been an attractive fuzzing target. The encoder operates on the pixel layout produced by the decoder. This is interesting because encoder routines are generally not designed to defensively handle inconsistent internal state originating from malformed decoder outputs. We ultimately prioritized the decoder because it is more directly exposed to attacker-controlled inputs in real-world deployments. We also considered the simplified high-level API (`png_image_begin_read_from_memory`), which is commonly used in modern applications due to its ease of integration and abstraction over the lower-level decoding pipeline, making bugs here high-impact. However, this API was introduced in `libpng 1.6.0` and is therefore unavailable in our target version, `libpng 1.2.52`.

2.2 Data Flow and Guards

The harness accepts input via `stdin` (instrumented campaign, no `@@` in the `afl-fuzz` invocation) or as a file path in `argv[1]` (QEMU campaign, invoked with `@@`). Input is read into a heap buffer, and a custom I/O callback (`read_fn`) is registered via `png_set_read_fn` so that `libpng` reads from memory rather than a `FILE*`.

A key design decision was synthetic signature injection: the callback prepends the valid 8-byte PNG signature before

splicing in fuzz data. This prevents AFL++ from wasting mutations on the fixed signature bytes, which libpng validates immediately and which yield no additional coverage when corrupted. An earlier version rejected invalid signatures using `png_sig_cmp`, but this caused rapid coverage stagnation because mutations affecting the signature never reached the parser.

The harness executes both the pull API (`pngread.c`) and push API (`pngpread.c`) on every input, as the two parsers share little implementation logic and expose distinct attack surfaces. The pull model targets synchronous file parsing, while the push model exercises incremental streaming logic where chunk boundaries may appear at arbitrary offsets. Restricting fuzzing to a single parser would therefore leave half the attack surface untested. To maximize decoder coverage, five transformation routines are enabled in the pull path, followed by `png_read_update_info()` to apply the registered transforms to the info struct so that `rowbytes` reflects the post-transform row size.

A `setjmp` recovery block surrounds all libpng calls because the library reports errors through `longjmp`; without a valid jump target, malformed inputs would appear as crashes and flood AFL++ with false positives. Finally, dimension guards (`height > 5000` or `rowbytes > 10,000,000`) prevent pathological allocations from mutated headers, while validation of `fread()` results avoids harness-induced memory errors unrelated to libpng itself.

3 Q2 — Instrumentation and Sanitizers

libpng was compiled with `afl-clang-fast`, `-fsanitize=address -g -O1`, and `-disable-shared`.

`afl-clang-fast` inserts AFL++ edge-coverage callbacks at compile time. Without it, AFL++ cannot observe which branches a test case exercises and falls back to blind mutation.

`-fsanitize=address` is what made crashes visible. The bug in `png_inflate()` is a stack-use-after-scope: without ASan the invalid stack read produces no signal. This was not theoretical; our no-sanitizer fork campaign (Q8, 8.83M execs) and QEMU campaign (Q7, 1.78M execs) both found zero crashes.

`-O1` over `-O0`: we chose `-O1` instead of `-O0` after evaluating their impact through identical 1-hour campaigns using a simplified harness. The `-O0` configuration achieved an average throughput of approximately 220 exec/sec, decreasing to around 130 exec/sec by the end of the run, whereas `-O1` maintained 290.2 exec/sec. This corresponds to an approximately $1.3\times$ slowdown under `-O0`, without any observable coverage improvement. Additionally, both configurations identified the same root cause across 15 crashes.

`-disable-shared` statically links libpng into the harness,

ensuring the instrumented library code is what AFL++ actually sees.

CRC patch. We applied

`utils/libpng_no_checksum/libpng-nocrc.patch`, neutralizing `png_crc_finish()` in `pngutil.c`. Every mutated chunk fails its CRC-32 check: critical chunks abort via `png_error()`; ancillary chunks are silently discarded. Either way, mutated bytes never reach the parser. Without the patch, coverage plateaus immediately. We applied it to both builds so the instrumented/QEMU comparison isolates emulation overhead only.

4 Q3 — Seed Corpus and Dictionary

4.1 Seed Corpus

We assembled a corpus of 32 PNG seeds covering the major axes of format variation that libpng branches on internally, split into two groups. The first 17 files were hand-crafted to target specific code paths: `corrupt.png` immediately exercises error-handling paths; `minimal.png` (8-byte signature only) hits the earliest abort paths; and four seeds were programmatically generated to target auxiliary chunk handlers — `seed_gama.png` carries a `gAMA` gamma chunk (triggering the file-gamma code path instead of the fallback), `seed_unknown.png` embeds a `fuZz` ancillary chunk (exercising `png_set_keep_unknown_chunks`), `seed_ztxt.png` places a zlib-compressed `zTXt` chunk after `IDAT` (reaching `png_read_end` rather than `png_read_info`), and `seed_itxt.png` carries an uncompressed UTF-8 `iTXt` chunk; the remaining 11 cover the core format variants (grayscale, RGBA, palette, 16-bit depth, Adam7 interlacing, `tRNS`, `tEXt` chunks, and extreme dimensions). The remaining 15 seeds were drawn from the official libpng test suite (`contrib/pngsuite/`), a collection of reference PNG files used by the libpng project itself for regression testing. Keeping all seeds under 200 bytes minimises calibration overhead and maximises executions per second.

4.2 Dictionary

We extended the AFL++ PNG dictionary with three token categories. *Chunk type names* (`IHDR`, `IDAT`, `IEND`, `PLTE`, `iCCP`, `tEXt`, `tRNS`, etc.) are the *terminal symbols* of the PNG chunk grammar; the 4-byte sequences on which libpng’s dispatcher branches. Without them the fuzzer must rediscover each token by random mutation with probability 2^{-32} per position. *Boundary length fields* (big-endian 4-byte values 0, 1, 13, $2^{32}-1$) stress length-parsing arithmetic. *IHDR sub-field values* (bit depths 1/2/4/8/16, color types 0/2/3/6, interlace flags 0/1) help AFL++ generate structurally valid headers faster, reaching the deeper pixel-data pipeline.

From the AFL++ status screen after 33 minutes hour and 1.71M executions, the dictionary shows very limited usage

(14 active entries out of 17.4k–17.7k available), indicating that most dictionary tokens did not significantly contribute to execution guidance. In contrast, the majority of new coverage was driven by havoc and splice mutations, which accounted for most of the exploration of new paths. This suggests that the fuzzing campaign was primarily mutation-driven rather than dictionary-guided, likely because the existing corpus already covered most structured input requirements.

5 Q4 — Campaign Analysis

We ran the instrumented campaign for 1 hour via `make fuzz` with `interesting_seeds` and the extended dictionary. All plots and the status screen are in Appendix A (Figures 1–5).

Key metrics after 1.71M executions: stability 100%, corpus count 1201, map density 32.85%, count coverage 3.72 bits/tuple, cycles done 6, exec speed 855 exec/sec, 24 crashes saved. During fuzzing, 314 new edges were discovered, corresponding to previously unseen control-flow transitions that expanded the coverage map over time. Stability at 100% confirms the harness is fully deterministic.

Edges curve (Figure 2): the campaign started at ~300 edges and grew to ~1250, showing three distinct phases; rapid discovery in the first 600s, a slower discovery phase with diminishing returns around 900 edges from $t=600$ –1,400s, and a final near-plateau starting around $t=1,400$ s where edge growth becomes minimal.

With 427 pending items and only 6 cycles completed, the campaign had not reached full saturation at the 33-minutes mark. The 32.85% map density indicates significant unexplored edges.

6 Q5 — Crash Triage

AFL++ found 24 unique crashes within the first 33 minutes, all sig:06 (SIGABRT). Triaging all 24 confirmed a single root cause, triggered via two chunk handlers: `png_handle_zTtXt` and `png_handle_iCCP`, both reaching the bug through the shared path `png_decompress_chunk` → `png_inflate`.

Reproduction. The crashing input was reproduced directly from the AFL++ crash corpus.

Minimization. We ran `afl-tmin`, reducing the input from 104 to 50 bytes (37.5% reduction, 196 executions), simplifying root-cause analysis while preserving the crash trigger.

Root cause. The bug is a stack-use-after-scope in `png_inflate()` (`pngutil.c:306`). A local buffer `umsg[52]` is declared inside an inner scope block and `msg` is pointed at it. When the block closes, ASan poisons the

stack memory; `png_warning(png_ptr, msg)` then reads the dead pointer. The full ASan stack trace is in Figure 6.

Fix. The bug is present in libpng 1.2.52 and absent in 1.2.53. Comparing the two versions, `png_inflate()` was restructured so that `png_warning` is called inside the scope block where `umsg` is declared, rather than after it closes. The exact commit is part of the 1.2.53 beta development cycle (February 2015).

In our experiments, the issue produced no observable crash behaviour without ASan instrumentation. Because the invalid access is limited to a short-lived stack read immediately after scope exit, we believe the issue is likely low severity in practical exploitation terms. Nevertheless, it remains real undefined behaviour and may still lead to unstable behaviour or limited information disclosure depending on compiler optimizations and stack layout.

7 Q6 — Attack Surface Analysis

Mozilla Firefox uses libpng to decode PNG images embedded in web pages. A remote attacker can serve a crafted PNG from any origin; the browser fetches and decodes it automatically without user interaction, making this a zero-click attack surface reachable from the open web.

Android (Skia graphics library) links libpng to decode PNG assets in applications and system UI. A malicious image delivered via MMS, email attachment, or file download could trigger a decoder bug within the receiving process. If the vulnerable process runs with elevated permissions (as system UI processes often do), this could facilitate privilege escalation.

Unexercised code paths. Despite the inclusion of the Progressive API and basic transformations, significant gaps remain in the harness’s coverage. First, the harness completely ignores the encoding pipeline (`pngwrite.c`). While the decoder is the primary entry point for external data, many server-side applications ingest user images, modify them, and then re-encode them. Vulnerabilities in the encoder—such as integer overflows during chunk length calculation or CRC generation—could allow an attacker to compromise a server during the output phase of an image processing pipeline.

Furthermore, the harness does not invoke `png_set_interlace_handling()`, leaving the complex coordinate-mapping and row-buffer reuse logic for Adam7 interlacing in `pngutil.c` unexercised. In legacy versions like libpng 1.2.52, these multi-pass calculations lack modern bounds-checking, making "off-by-one" errors a high-risk source of heap corruption.

8 Q7 — Binary-Only Fuzzing (QEMU Mode)

We compiled the harness with `gcc` (no AFL++ instrumentation, no sanitizers) and ran AFL++ with `-Q`, using the same seed corpus and dictionary as the instrumented campaign.

Coverage mechanism. In compile-time instrumentation mode, AFL++ inserts inline callbacks at every branch edge via `afl-clang-fast`, updating a shared bitmap in $O(1)$ per edge with near-zero overhead. In QEMU mode, AFL++ runs the binary under a modified QEMU user-mode emulator that translates each basic block from x86 machine code into an intermediate representation on first encounter, injects coverage tracking, and caches the result. This translation happens at runtime, adding significant per-instruction overhead compared to compile-time instrumentation.

Table 1 compares both campaigns after 33 minutes. Plots for the QEMU campaign are in Figures 7–11.

The edges curve (Figure 8) started at ~ 800 and grew continuously to $\sim 3,200$ over the 33 minutes with no plateau, contrasting sharply with the instrumented campaign’s three-phase curve. This reflects that QEMU completed only 3 cycles with 61 pending items. The corpus grew steadily to 1047 entries (Figure 9), and the pending items curve started declining towards the end meaning the fuzzer had reached the plateau and there was nothing more interesting left to explore. Exec speed (Figure 11) averaged ~ 890 exec/sec early in the run, declining slightly to ~ 300 exec/sec near the end as the corpus declined. The crash/levels graph (Figure 10) shows zero crashes and zero hangs throughout, with queue depth growing to 20 levels.

Exec speed was surprisingly higher for QEMU than the instrumented campaign. This is unusual but could be explained by the fact that we ran our campaign on a cluster machine which has much higher memory and CPU throughput. In particular, AddressSanitizer introduces significant shadow-memory overhead that is highly memory-bandwidth intensive; on high-performance hardware, this cost becomes comparatively less dominant. In contrast, QEMU mode remains primarily CPU-bound due to dynamic binary translation overhead, which scales less effectively with increased memory bandwidth.

Edge counts were similar in absolute terms (314 vs. 281) but map density differed dramatically (32.85% vs. 4.83%): compile-time instrumentation produces a denser, more accurate coverage map, while QEMU’s runtime translation produces a sparser approximation.

Crashes. The QEMU campaign found no crashes after 33 minutes. The bug is *sanitizer-dependent*: the `stack-use-after-scope` produces no OS signal without ASan, and QEMU mode can only detect crashes generating SIGSEGV or SIGABRT.

9 Q8 — Instrumentation Depth and Performance

9.1 Instrumented Edge Counts

Using `AFL_DEBUG=1`, the libpng library shows multiple instrumentation outputs across its compilation units, with values

ranging from approximately 13 to 1153 instrumented locations per file. Aggregating across all files, this corresponds to several thousand instrumented edges in total.

For the final fuzzing binary, the harness instrumentation reports 53 instrumented locations with no collisions. In addition, AFL++ reports approximately 3803 coverage edges in the final linked executable, corresponding to the full merged coverage map of the harness and all linked libpng code.

These numbers differ because libpng is a large image decoding library with many functions, branches, and error-handling paths, resulting in thousands of instrumented edges; the harness is a minimal wrapper that only calls a small subset of libpng APIs, producing only 53 instrumented locations; and the final value of 3803 represents the total AFL++ coverage map size for the fully linked executable, including the harness, libpng, and all linked runtime components.

9.2 Exec Speed Comparison

Table 2 compares exec speed across three configurations, all using the same harness and seed corpus. Plots for the no-sanitizer and persistent campaigns are in Figures 12–16 and Figures 17–21 respectively.

No sanitizer vs ASan + fork (4415 vs 855 exec/sec):

Disabling ASan yielded a $5.16\times$ speedup by removing shadow-memory checks on every memory access, leaving only AFL++ coverage instrumentation overhead. The no-sanitizer campaign completed 21 cycles with one pending item, compared to six cycles and 427 pending items for ASan + fork mode. However, despite 8.83M executions, it detected no crashes, showing that the bug was only observable with sanitizer instrumentation.

ASan + persistent mode (5450 exec/sec, $6.37\times$ speedup):

Persistent mode eliminates repeated `fork()` overhead by executing the harness loop 10,000 times per process via `__AFL_LOOP(10000)`. This increased executions from 1.71M to 10.9M over the same runtime. The campaign completed 17 cycles with zero pending items, fully exhausting the queue.

The edges curve (Figure 18) exhibits a rapid increase to approximately ~ 1200 edges by $t = 300$ s, followed by a much slower rise to about ~ 1260 by $t = 1200$ s. After this point, the curve remains flat, indicating that the campaign reached full saturation in under 20 minutes. In contrast, the ASan + fork campaign had not yet saturated after 30 minutes.

The exec speed graph (Figure 21) shows 5,000–8,000 raw exec/sec. The corpus/pending graph (Figure 19) shows pending items spiking to ~ 220 then dropping to near zero by $t=600$ s; the queue was exhausted in under 10 minutes. 77 unique crashes were found (Figure 20), mostly within the first 1,800s, confirming ASan is active. Stability was 99.92%, a slight drop from fork mode’s 100%, expected in persistent mode due to minor state accumulation across iterations.

A Instrumented Campaign (Main, Q4)

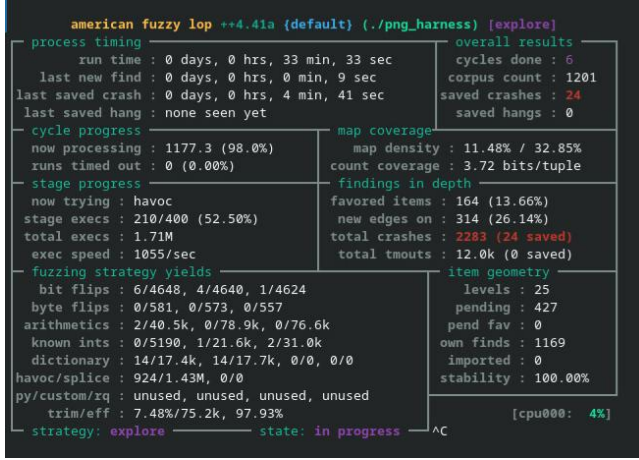


Figure 1: AFL++ status screen — instrumented campaign (ASan + fork, 33 minutes). 850 exec/sec, 24 crashes, 6 cycles, queue not exhausted.

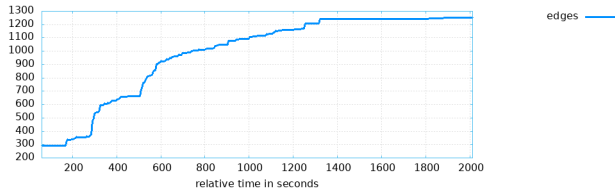


Figure 2: Edges graph — instrumented campaign. Edge count grows from ~300 to ~1250 with distinct growth phases.

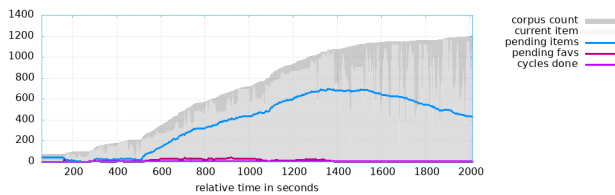


Figure 3: Corpus and pending items — instrumented campaign. Corpus grows to 1201; pending items decline as the queue is processed.

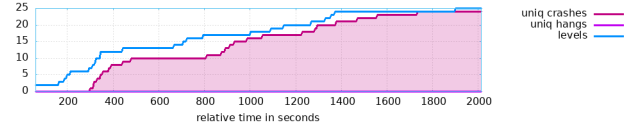


Figure 4: Crashes and queue levels — instrumented campaign. 24 unique crashes found; queue depth reached 25 levels.

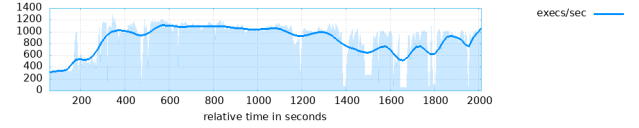


Figure 5: Execution speed — instrumented campaign. Average ~855 exec/sec throughout the run.

B ASan Stack Trace (Q5)

```
==ERROR: AddressSanitizer: stack-use-after-scope
READ of size 1 at 0x7ffdef56d3a0
#0 png_default_warning pngerror.c:318
#1 png_warning          pngerror.c:132
#2 png_inflate          pngutil.c:306
#3 png_decompress_chunk pngutil.c:338
#4 png_handle_iCCP      pngutil.c:1122
#5 png_read_info        pngread.c:529
#6 main                 harness.c:86
Variable 'umsg' (line 284) <== Memory access
is inside this variable
```

Figure 6: ASan stack trace for the triaged crash (iCCP variant).

```
/* 1.2.52 - buggy */
msg = umsg;
} /* umsg destroyed */
png_warning(ptr, msg); /* reads dead stack */

/* 1.2.53 - fixed */
msg = umsg;
png_warning(ptr, msg); /* inside scope */
}
```

C QEMU-Mode Campaign (Q7)

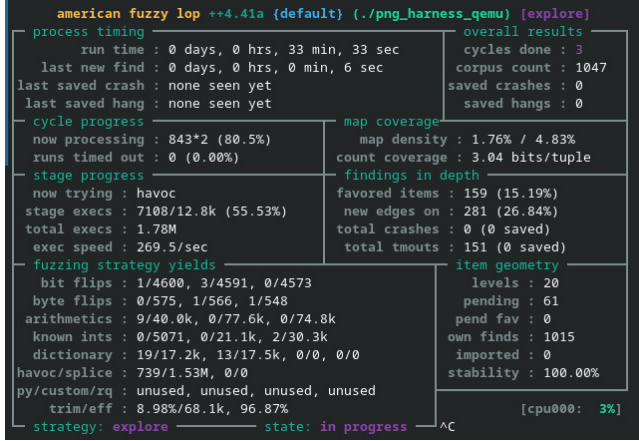


Figure 7: AFL++ status screen — QEMU campaign (33 minutes). 890 exec/sec, 0 crashes, 3 cycles.

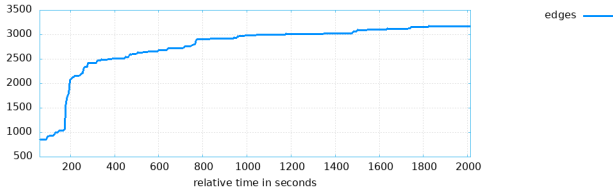


Figure 8: Edges graph — QEMU campaign. Edge count grows from ~800 to ~3,200 with two distinct growth phases: a sharp increase followed by a slight increase.

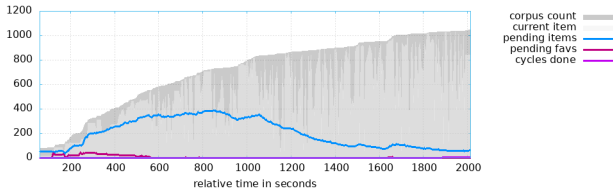


Figure 9: Corpus and pending items — QEMU campaign. Corpus grows to 1047; pending items declined to near zero by end, indicating full queue processing.

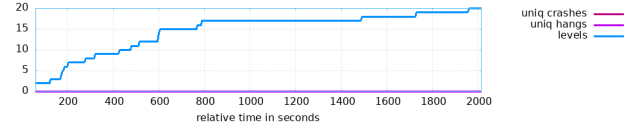


Figure 10: Crashes and levels — QEMU campaign. Zero crashes and zero hangs throughout; queue depth reaches level 20.

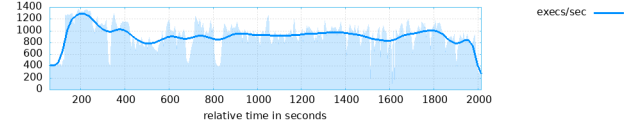


Figure 11: Execution speed — QEMU campaign. Average ~890 exec/sec throughout the run.

D Table QEMU vs Instrumented

Metric	Instrumented	QEMU
Exec speed	850/sec	890/sec
Total execs	1.71M	1.78M
New edges	314	281
Corpus	1201	1047
Map density	32.85%	4.83%
Crashes	24	0
Cycles done	6	3

Table 1: Instrumented vs. QEMU after 0.5 hour.

E No-Sanitizer Campaign (Q8)

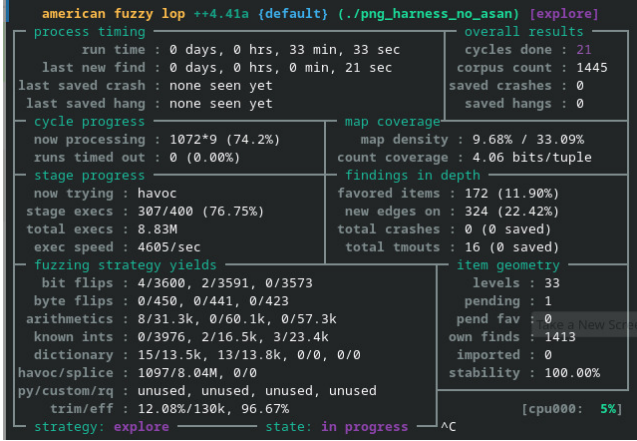


Figure 12: AFL++ status screen — no-sanitizer campaign (fork mode, 1 hour). 4415 exec/sec, 0 crashes, 21 cycles completed.

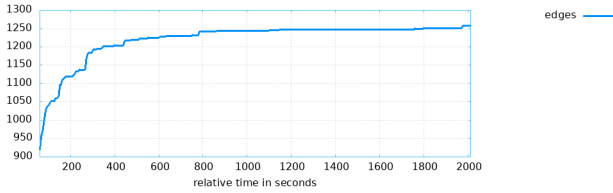


Figure 13: Edges graph — no-sanitizer campaign. Growth from ~900 to ~1250, plateauing after $t=800$ s.

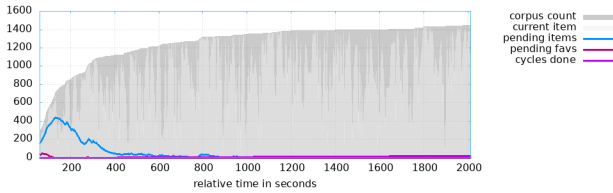


Figure 14: Corpus and pending items — no-sanitizer campaign. Pending items declined to zero by $t=1000$ s, indicating full queue processing.

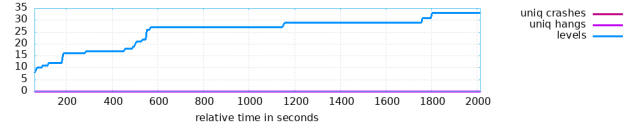


Figure 15: Crashes and levels — no-sanitizer campaign. Zero crashes throughout; queue depth reached 33 levels.

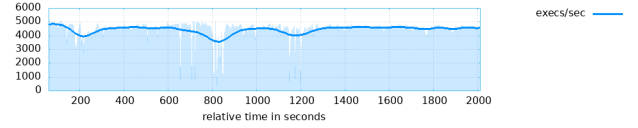


Figure 16: Execution speed — no-sanitizer campaign. Average ~4415 exec/sec.

F Persistent Mode Campaign (Q8)

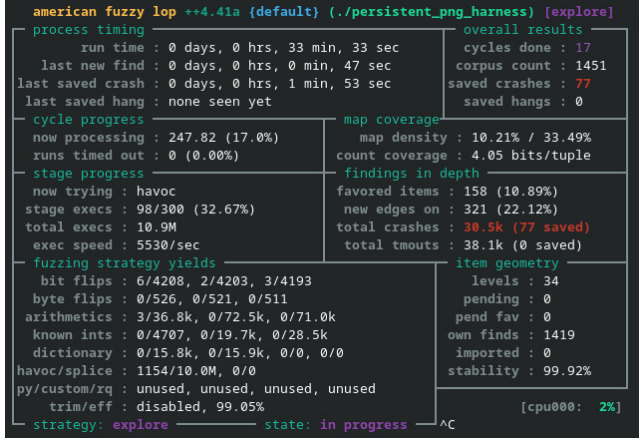


Figure 17: AFL++ status screen — persistent mode campaign (ASan, 33 minutes). 5450 exec/sec, 77 crashes, 17 cycles, queue fully exhausted.

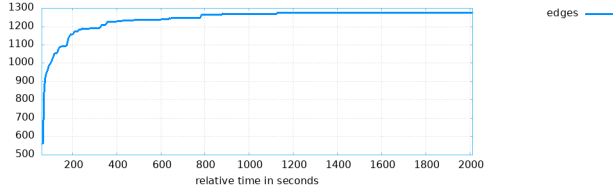


Figure 18: Edges graph — persistent mode. Full saturation reached at $t=1,200s$ (20 minutes); flat thereafter.

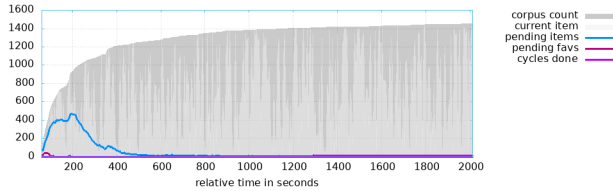


Figure 19: Corpus and pending items — persistent mode. Pending items exhausted by $t=600s$; 17 cycles completed.

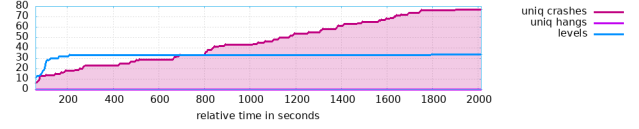


Figure 20: Crashes and levels — persistent mode. 77 unique crashes found, mostly within first 1,800s.

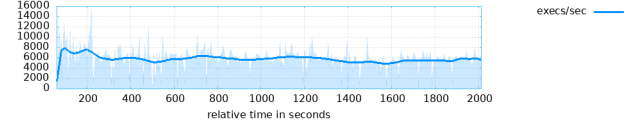


Figure 21: Execution speed — persistent mode. Average $\sim 5,450$ exec/sec throughout the run.

G Comparison Table (Q8)

Configuration	Exec/sec	Total execs	Speedup
No sanitizer + fork	4415	8.83M	5.16×
ASan + fork (baseline)	855	1.71M	—
ASan + persistent	5450	10.9M	6.37×

Table 2: Exec speed across three configurations (same seeds, same harness, 33 minutes).